
AGENTIC AI UNDER A FIVE-HOUR DEADLINE: TRACES OF CODING AGENTS IN 3,649 HACKATHON REPOSITORIES

PREPRINT

Talgar Bayan^{1,2}

¹School of Computing and Artificial Intelligence (SCAI), Nazarbayev University, Astana, Kazakhstan

²University of Manchester, Manchester, United Kingdom

talgar.bayan@gmail.com

3 October 2026

ABSTRACT

The adoption of AI coding agents is often measured from their traces in repositories, usually without knowing which repositories could have shown activity or which tool developers were asked to use. We study HackAlem AI, a five-hour agentic AI hackathon in Astana on 23 September 2026, powered by OpenAI, whose rules required teams to use Codex without excluding other tools. The organisers created a public GitHub repository for registered teams in advance. We mirrored all 3,649 repositories and recounted core counts with separate code. Organiser actions shaped the population: after early archiving and an unused evening batch, event-time work reached 29.0% of all repositories but 48.2% of the open repositories outside the batch. Codex-specific traces appeared in 21.7% of active repositories, or 42.0% counting the shared AGENTS.md, and a web framework had written such files itself in 8.4%. Among teams naming the tool in their README (exploratory), signatures revealed Claude Code in 73.2% and Codex in 3.3%. Teams given the same brief chose more similar libraries. OpenAI key strings reached the public history of 61 repositories, mostly through environment files. Repository-based measures of agent use therefore depend on how the population was formed and on tool and framework defaults.

Keywords AI coding agents · agentic AI · mining software repositories · hackathons · OpenAI Codex · credential leakage

1 Introduction

AI coding agents such as OpenAI Codex and Claude Code edit whole repositories from natural-language instructions; in 41% of the real sessions collected in one dataset, the agent wrote virtually all of the committed code [1]. How widely such agents are used, and with what effect, is increasingly measured from the traces they leave in version control: co-author trailers in commit messages, pull requests and context files such as AGENTS.md [2, 3]. Large studies built on these traces report adoption rates, productivity gains and quality costs [2, 4, 5], and their conclusions depend on how completely the traces record agent use.

Two quantities are usually unknown in this research. The first is the denominator: the set of repositories that could have shown activity, including those that were never used, which samples drawn from project listings or from detected agent activity cannot contain [6, 7]. The second is the tool the developers were meant to use. An agent that does not sign its commits leaves no signature, so a repository without a trace may belong to a team that used an agent silently, and heuristics for detecting agents from their traces have known pitfalls [8].

Hackathons offer a setting in which both quantities can be fixed in advance. They are time-bounded events at which small teams build prototypes [9], and their repositories have been studied at scale, mostly from projects that teams chose to list on platforms such as Devpost [10, 11]. Studies of generative AI at hackathons have so far examined single events through interviews, surveys and project evaluations [12, 13]. HackAlem AI, an agentic AI hackathon held in Astana on 23 September 2026 and powered by OpenAI, differed in two respects. Before the event, the organisers created a public

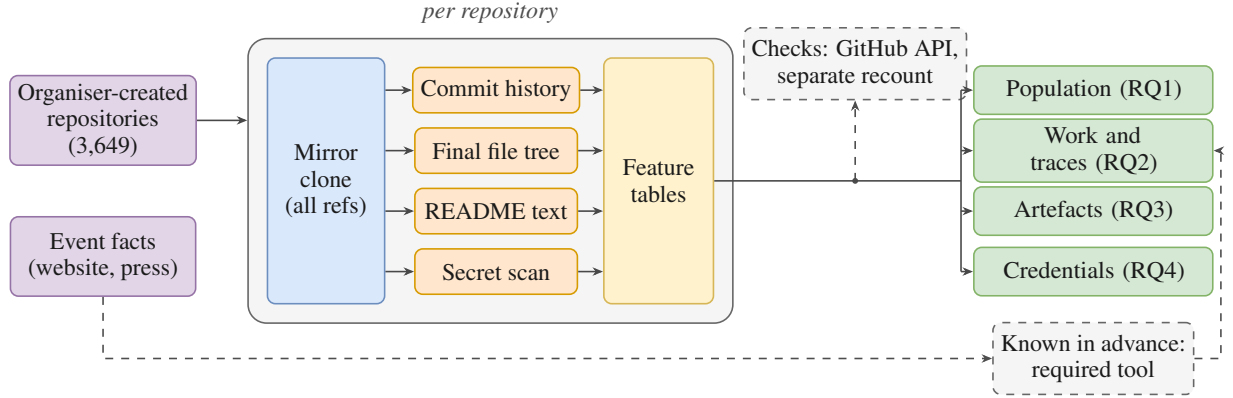


Figure 1: Study architecture. Every organiser-created repository was mirrored with all its references and passed through the same pipeline into feature tables; the secret scan covers the repositories with team commits. The tables were checked against the GitHub API and a separate recount, and four analyses use them. The event facts fix the required tool, against which the agent traces are compared; the event window is inferred from the data (Section 3.2).

GitHub repository for registered teams, so the GitHub organisation holds repositories that never received a commit as well as those that did. Its rules also required every team to use Codex, OpenAI’s coding agent, during development, without excluding other tools [14]. The tool that the traces of every team should reveal is therefore known, and traces of other agents show what teams used beyond it.

We use this setting to examine what repository traces show about agent-assisted work when the denominator and the intended tool are known. Our argument is that rates measured from repositories depend on how the population of repositories was formed and on the default behaviour of each tool, and that both have to be accounted for before such rates are read as evidence of how people work with coding agents. We mirrored all 3,649 repositories with their full history, 31,955 team commits in all, and use this complete record of one event to ask four research questions. The first (RQ1) asks how many repositories carried work during the event and how much of the gap reflects the organisers’ own actions. The second (RQ2) asks how teams spread their work over the five hours and which traces the required agent and other agents leave, including among teams that name the tool in their README. The third (RQ3) asks what teams built, how teams given the same brief solved it and how they documented their work, and the fourth (RQ4) how credentials reached the public history. Figure 1 shows the study design.

Of the 2,592 repositories without event-time work, 1,472 had been archived before the event or created in an evening batch that stayed almost unused. Traces specific to Codex, the required tool, appeared in about a fifth of the active repositories, a web framework had written agent context files of its own in 8.4% of them, and among teams that named each tool, commit signatures revealed Claude Code, which signs by default [15], far more often than Codex.

The paper makes four contributions. First, it reports participation on a known denominator and separates the organisers’ actions from the teams’ activity, a step that studies of organiser-created repositories have to take before any rate can be read. Second, it measures how often the trace heuristics used in agent-adoption studies detect the intended tool when that tool is known, and how detection differs between a tool that signs its commits by default and one whose signatures were rare; it also shows that the web framework Next.js writes the context files of two agents itself, which inflates counts of such files. Third, it gives population-level estimates, with intervals, of the artefacts, documentation and exposed credentials of prototypes built under an agent requirement and one deadline, and it shows that teams given the same brief converged on similar solutions, with library overlap not clearly greater among teams that left agent traces. Fourth, it releases a pseudonymised dataset and a pipeline in which every number is generated from the data, the core counts are reproduced by separate code, and the headline results can be recomputed from the public files alone.

Section 2 reviews related work, and Section 3 describes the event, the data, the measures, the threat model and the statistics. Section 4 presents the results. Section 5 discusses them, Section 6 examines threats to validity, and Section 7 concludes.

2 Background and related work

2.1 Hackathons and their repositories

Hackathon research spans the purpose, format, processes and outcomes of events [9]. Repository studies have related project continuation to preparation, winning and team skills [16], found that most commits of hackathon projects fall in the first week and that few projects remain active months later [17], and measured how much code is written during an event and reused afterwards, starting from 22,183 projects listed on Devpost [10, 18]. HackRep collects 100,356 hackathon repositories and shows that event durations can be estimated from commit timestamps [11]. Samples of listed projects cannot contain registrations that produced no code. General cautions for mining GitHub apply to all of these datasets: many repositories are inactive or personal [6], git histories can be rewritten and their timestamps are set by clients [19], mining tools disagree on basic counts [20], and random sampling is rare in software engineering research [7].

2.2 Measuring coding-agent use from repositories

Before agents, ChatGPT use in open-source projects was found by matching messages that mention it and checking each match by hand [21]. Agents leave more explicit traces. The AIDev dataset collects pull requests authored by five agents [22], and studies of projects that adopted agents report velocity gains that were transient [5] or confined to projects for which the agent was the first AI tool [4], together with lasting increases in code complexity and static-analysis warnings [4, 5]. Robbes et al. use traces such as co-authored commits and pull requests to estimate that 22.20–28.66% of 128,018 GitHub projects had adopted a coding agent by February 2026, find that commits assisted by agents are larger than commits written by developers alone, and describe the heuristics and pitfalls of detecting agents from traces [2, 8]. In open-source projects, agent context files such as `AGENTS.md` and `CLAUDE.md` change like configuration code and mostly describe tests, implementation and architecture [3]. Codex reads `AGENTS.md` before doing any work [23], the file is an open format that many agents read [24], and in a benchmark such context files did not improve task success on average while raising inference cost [25]. Benchmarks measure what agents can do on repository tasks [26, 27], not how often or how visibly they are used. The field studies observe agents in projects whose developers chose their tools freely, so the share of agent use that the traces miss cannot be estimated from them.

2.3 Generative AI at hackathons and under time pressure

In a controlled experiment, developers with GitHub Copilot finished a task faster than a control group [28], and in a field study perceived productivity was linked to how often suggestions were accepted [29]. Hackathons have since become a setting for studying such tools. At a two-day AI-themed hackathon, participants used generative AI for coding, learning and documentation, combined several tools, and checked the output under time pressure [12]. At a one-day educational hackathon, novices who built software mainly by prompting, often called *vibe coding*, delivered working demonstrations but converged early on ideas and engaged little with engineering practice [13]. In a month-long online hackathon, a rule against manual editing shaped how participants prompted and debugged [30], and a university case discusses the educational opportunities and risks of such tools [31]. Computing educators have also reviewed what these tools mean for teaching [32].

On the use of time, commit timestamps show when developers work [33], students who start earlier tend to finish earlier and score higher [34], and most high-quality studies of time pressure report higher productivity and lower quality [35].

2.4 Credentials in public repositories

Secrets often leak to public GitHub repositories: a scan of public commits in real time over nearly six months found thousands of new, unique secrets leaked every day [36]. A labelled dataset supports the evaluation of detectors [37], and a comparison of detectors found Gitleaks to have the highest recall (88%) with lower precision (46%) [38]. Developers who leaked secrets describe prevention and remediation as hard [39]. Participants with an AI assistant wrote less secure code while believing it more secure [40], and coding agents produced functionally correct but insecure solutions to real-world tasks [41]. LLM API keys also leak from mobile apps, and most of the reported leaks remained exploitable months after disclosure [42]. GitHub now reports keys of partner services found in public repositories, OpenAI API keys among them, to the provider, which decides whether to revoke them [43, 44], and by default it blocks each user’s pushes of such keys to public repositories, although the block can be bypassed [45].

Table 1: Facts about the event. Organiser and news sources were read on 24 September 2026 or later; the event window is our inference.

Item	Value	Source
Date and place	23 September 2026, Astana, in person	[14]
Duration	five hours	[46–48]
Event window	13:00–18:00 local time (UTC+5)	this study
Teams	up to three members; individual registration allowed	[14]
Format	agentic AI hackathon; competition between teams; powered by OpenAI, a partner of the event	[14]
Tooling rule	teams must use Codex during development; other tools not excluded	[14]
Eligibility	aged 18 or over; working in IT, studying IT or a recent IT graduate, or with own software projects	[14]
Capacity	2,500 seats, first come, first served; 6,586 applications	[14, 48, 49]
Participants	more than 2,500 from 21 countries and 129 universities	[46]
Record attempt	Guinness World Records, most participants in an agentic AI hackathon; previous record 2,089; result pending at the time of writing	[46]
After the event	jury review of submitted projects 24 September–13 October; finalists present and winners are announced on 16 October; awards on 23 October 2026	[14]
Repositories	public repositories in BAITC-Hacks, one per registration (assumed)	this study

2.5 Positioning of this study

Trace-based studies of agents measure use in samples whose selection and intended tools are unknown, hackathon studies based on repositories see only listed projects, and studies of AI at hackathons rely on what participants and judges report rather than on the repositories of every registered team. Credential leakage has been measured across GitHub, but not for software built under an agent requirement and a common deadline. We found no repository-level study of a hackathon in which repositories were created for registered teams in advance and one agent was required. That setting allows the traces used to measure agent adoption to be compared with a known intended tool, and participation, work, artefacts and exposure to be measured on a known denominator.

3 Materials and methods

3.1 Study setting

Table 1 lists the facts about the event that we use, with their sources. HackAlem AI described itself as an agentic AI hackathon, a competition between teams, and was powered by OpenAI, one of its partners. Its rules required teams to use Codex during development and did not exclude other tools [14]. Participation was open to people aged 18 or over who worked in IT, studied an IT subject or had graduated from full-time IT study in the previous 12 months, or had software projects of their own. Registration was free, people could register alone, and the 2,500 seats were given on a first-come, first-served basis [14]. News reports describe long queues at the entrance and registered participants who could not get in, and the organisers said that participants had been told in advance that seats were limited [48, 49]. The event had 12 tracks, each with a brief, a task statement set by an industry or public-sector partner (Section 4.4). Each repository was named `hack-id-team` and was created with an organiser commit titled “Initial commit” that added a README of two lines. The organisers then archived the repositories, which made them read-only: 3,201 were archived between 18:01 and 18:51 on the event day and 2 later that evening, and the remaining 446 had been archived before the event (Section 4.1). A repository is *open* if it was not archived before the event.

3.2 Data collection and definitions

On 24 September 2026 we listed all repositories of the organisation through the GitHub REST API, including their creation and last-update times, collected default-branch commit counts through GraphQL, and made mirror clones of all 3,649 repositories. A mirror clone copies every reference of a repository, that is, every branch, tag and pull-request reference, with the full history. All clones completed, and because the repositories were archived their content could no longer change. All measures are computed by deterministic scripts, from the clones or, for the track labels, from an earlier scan of the same repositories. The one exception is the coding of each team’s approach, for which a language model read the README (Section 3.3).

A *team commit* is a commit reachable from any branch, counted once per SHA, other than the organisers’ template commit. We identify the template commit by a rule that does not depend on the author account: a commit without parents, titled exactly “Initial commit”, made within 120 s of the repository’s creation, that adds two lines to one file and

deletes none. The *event window* is 13:00–18:00 local time on 23 September 2026, measured on committer timestamps; we inferred it from commit and archive times together with press reports of a five-hour event, since we found no official schedule. An *active repository* has at least one team commit in the window. For an active repository i with c_i window commits, of which ℓ_i fall between 17:00 and 18:00, the *last-hour share* is $s_i = \ell_i/c_i$; an even spread of work over the five hours would give 20%. The *hours with commits* of a repository are the number of the five clock hours of the window in which it received at least one team commit. We read a last-update time before the start of the window as *archived before the event*, because archiving updates this time and none of these repositories received a commit during the event. The *evening batch* consists of the repositories created during 18:00–20:00 on 22 September; we treat it as an organiser action, because up to 15 repositories were created per minute then, against at most 8 in any minute of earlier days.

A repository stands for one registration of up to three people, and its creation date stands in for the registration date; we make no claims about individuals. The population analysis uses all 3,649 repositories. All other analyses use the 1,057 active repositories; README measures use the 1,033 of them with a written README, that is, a README other than the organisers’ two-line template with at least 20 letters of prose, and commit measures use their team commits in the event window.

3.3 Measures

From the final state of each default branch we record the primary language by file size, frameworks and libraries from package manifests, and LLM use from manifests, import statements and API endpoints, excluding vendored folders, that is, copies of third-party code such as `node_modules` or virtual environments. Engineering artefacts are test files, a Dockerfile, a Compose file, a continuous-integration workflow and a deployment configuration. A test file lies in a folder named `test` or `tests` or has a name such as `test_x.py` or `x.test.ts`; as a stricter measure we record whether a test file in Python, JavaScript, TypeScript or Go defines a test, such as a function whose name starts with `test` or a call to it or `describe`. Track labels come from an earlier scan made for a public report on the event, whose script is released with the data. It scored the start of each README with keyword rules for each track, in which a phrase unique to the track, such as the title of its brief, outweighs all other keywords, and it left a repository unassigned when no track scored at least two points. We checked the labels against a second set of keyword rules written for this check. Where the rules gave one clear track, they agreed with the label for 933 of 946 active repositories (98.6%, $\kappa = 0.98$). We read every disagreement and corrected 7 labels. Of the labelled repositories, 19 had no README text to check, and 73 active repositories stayed unassigned, 34 of them although the keyword rules suggested a track.

For each agent we record four kinds of repository trace. A context file is `CLAUDE.md` or a `.claude` folder for Claude Code and the rule files of Cursor, Copilot and Gemini for those tools. For Codex it is `AGENTS.md` at any depth, a file that Codex reads [23]. Files in vendored folders such as `node_modules` are not counted, nor are the context files that the web framework Next.js writes itself: `create-next-app` writes an `AGENTS.md` holding only a block that Next.js manages, together with a `CLAUDE.md` that only imports it, and `next dev` writes both when it detects a coding agent [50], so these files do not show which agent, if any, was used. Many other agents read the same file [24], and current versions of Claude Code read it when a repository has no `CLAUDE.md` [51], so we report it separately from the traces specific to Codex. A branch trace is a branch whose name starts with the tool’s name. A commit signature is a co-author trailer (a Co-authored-by line at the end of the commit message) or a “generated with” line that names an agent, or an agent service account, that is, a GitHub account run by the tool’s vendor, as author or committer. An author trace is an author or committer named exactly after the tool, such as “Codex”, with no other signature on the commit. Which of these traces each way of using Codex (command line, editor extension or cloud tasks) leaves by default is not documented in the sources we read, so we do not attribute a trace to one of them. Generic bot accounts, among them the organisers’ application, are not counted. Pull-request references, which GitHub keeps for every pull request, are reported as a workflow trace, because teams create pull requests with or without agents. As a further reference we record whether a written README names Codex, or Claude Code, in its prose, that is, outside code blocks, file names, links and model names. A broad variant, which matches the word anywhere in the raw README, is reported for comparison. A README can be written by an agent, so naming the tool is not independent of the traces; we treat it as what teams chose to state, not as ground truth.

For the README at the repository root we remove code, links and markup and classify the remaining prose by its letters. A README is Kazakh if at least 50% of its letters are Cyrillic and at least 3% of those are among the nine letters that Kazakh has and Russian does not, Russian if at least 50% of its letters are Cyrillic but it does not meet the Kazakh condition, English if at least 80% of its letters are Latin, and mixed otherwise. The script of commit subject lines is classified in the same way. README content has been categorised in earlier work [52]; we record the length of the README, whether it has run or setup instructions (a heading on installation, setup or usage, or a line in a code block that starts with a command), images, and a link to a deployed application. For each non-merge window commit

we count the lines changed outside lock files, data files and generated output, so that the size of a commit reflects the code and text that were written, and whether its subject follows the conventional-commit form, a type prefix such as `feat:` or `fix:`.

To compare solutions to the same brief, we treat each track-labelled active repository as the set of dependency names in its package manifests and measure the similarity of two repositories by the Jaccard index of their sets; the same is done with the sets of file paths and of README headings. Pairs of repositories with the same track are compared with pairs with different tracks, with a one-sided permutation test that shuffles the track labels 999 times. The excess similarity of same-track pairs, their mean Jaccard index minus that of cross-track pairs, is then compared between pairs in which both repositories have an agent trace of any kind, including `AGENTS.md`, and pairs in which neither does, with a two-sided test that shuffles trace status within tracks. Source files of at least 200 bytes that occur unchanged in several repositories are found through their git object identifiers. To describe how each team solved its brief, a language model (deepseek-flash from DeepSeek, queried on 28 September 2026 at temperature zero, without its reasoning mode) coded every track-labelled active repository with a fixed codebook, given the brief, the first 2,500 characters of the README and up to 40 file paths. Links, e-mail addresses and code blocks were removed from the README, and no other data were sent to the model. The codebook asks for the core method (a prompt to an LLM, an LLM agent, a trained model, rules or an algorithm, a non-generative pretrained model such as speech recognition, or unclear when the README does not say), the role of an LLM (none, producing the main result, or support such as explaining a result), the interface, and whether the README reports a measured result. The diversity of methods within a track is summarised by the effective number of methods, the exponential of the Shannon entropy of their shares. The answers largely agree with the rule-based detection of LLM libraries: 88.2% of the repositories coded as using an LLM contain an LLM SDK or API call, and 81.3% of those coded as using none contain neither (Cohen’s $\kappa = 0.63$). We did not compare the codes with hand labels, so the coded approaches are not validated against human judgement and serve only to describe the solutions. The codebook, the prompt and all answers are released.

3.4 Credential scanning and threat model

We ran Gitleaks [53] version 8.30.1 with redacted output over every reference of the 1,254 clones with team commits. Its format-specific rules match the formats of particular keys and tokens, such as OpenAI API keys, JSON Web Tokens or credentials in `curl` commands, and its generic rule matches high-entropy strings assigned to names such as *key* or *secret*. For each finding we kept, privately, the rule, file, commit, line and the author date that Gitleaks reports, and whether a string matched by the same rule is still present in the same file in the final state. We never stored or printed a matched value and never tested one against a service. For format-specific rules, a string counts as a placeholder if it contains placeholder text (such as a run of `x`, “your”, “example” or angle brackets) or has a character entropy below 3.5 bits.

The threat model is that of an outside observer who can read every public repository and its full history, as a real-time scan of public commits can [36], but has no other access. The assets are credentials for paid services used during the event, mainly LLM API keys. A credential counts as exposed if a string in a format that these rules cover appears in any commit reachable from a public reference, whether or not it remains in the final version, because the history of an archived repository stays readable. We do not know whether a key was valid, revoked or used, since testing keys would itself be misuse, and GitHub reports OpenAI keys in public repositories to OpenAI, which may revoke them [43]. Within the formats that the rules cover, the counts are therefore upper bounds on usable keys; across all formats they are lower bounds on exposure, because keys in other formats are missed [38].

Author names and e-mail addresses were read only to match agent patterns and to count distinct authors through a salted hash whose salt was never saved; they were not stored. We informed the organisers privately on 28 September 2026, by e-mail and by a Telegram message, before publication; the e-mail listed the affected repositories and files without any key values and recommended that the keys be revoked.

3.5 Validation

Table 2 lists the checks. Default-branch commit counts from the clones equal those from the GitHub API for every repository. A second script, written separately from the pipeline and sharing no code with it, reproduced team commits, window commits and hours with commits for every repository; it guards against errors of implementation, not of definition. While spot-checking repositories on GitHub we found template commits made by a second organiser account; an audit of all 402 repositories this could affect found 16 such repositories, which the account-independent rule now handles. Commits reachable only through pull-request references (427 commits in 117 repositories) are not team commits. The number of active repositories hardly changes when the window is shifted or author time replaces committer time. The author labelled a sample of READMEs, stratified by the language rule’s output, without seeing

Table 2: Validation and sensitivity checks of the collected repositories and derived measures. Counts refer to the 3,649 organiser-created repositories unless stated otherwise.

Check	Result
Default-branch commit counts, clones vs. GitHub API	3,649 of 3,649 equal
Separate recount of team commits / window commits / hours with commits	3,649 / 3,649 / 3,649 of 3,649 equal
Template commit found by the account-independent rule	3,615; 34 rewrote their history
Template commits by a second organiser account	16 of 402 audited
Consistency with the earlier table (team / window commits, hours)	3,641 / 3,644 / 3,648 of 3,649 equal
Active repositories with author time instead of committer time	1,057 (committer time: 1,057)
Active repositories with the window shifted by $-60 / -30 / +30 / +60$ min	1,026 / 1,040 / 1,060 / 1,059
Track labels vs. keyword rules (active, one clear track)	933 of 946 agree; $\kappa = 0.98$
README language rule vs. the author’s blind labels (stratified sample)	69 of 80 agree; $\kappa = 0.80$

that output. Of 81 sampled READMEs, 80 could be labelled and one had no prose. The rule agreed with the labels for 69 of the 80 (86.3%, $\kappa = 0.80$), and for about 98% of written READMEs when each stratum is weighted by its share of them. All READMEs that the rule called Russian were confirmed (29 of 29); the errors lay in READMEs it called English (22 of 25 confirmed), Kazakh (16 of 20; the others were Russian) or mixed (2 of 6). One sampled README was partly in Chinese, a script the rule does not cover; no other written README of an active repository contained Chinese characters.

3.6 Statistical analysis

The repositories are the complete set for this event, so intervals and tests describe how precisely the observed shares reflect the process that produced them, not sampling from other events [7]. Proportions carry 95% Wilson confidence intervals (CI), agreement is measured with Cohen’s κ , and distributions are summarised by the median and interquartile range (IQR). We write $\chi^2(d)$ for a chi-square statistic and $H(d)$ for a Kruskal–Wallis statistic with d degrees of freedom; p -values are two-sided unless stated otherwise. Groups are compared with χ^2 tests, with Cramér’s V as the effect size and the number of cells with small expected counts reported, and with Mann–Whitney tests, with Cliff’s δ . One association is adjusted with logistic regression and one with a Mantel–Haenszel odds ratio. The odds ratios between naming a tool and having its traces add 0.5 to each cell and use Woolf’s log-scale intervals. Tracks are compared with a Kruskal–Wallis test, with the rank-based effect size $\varepsilon^2 = H/(n - 1)$ [54]. The sizes of agent-signed and other commits are compared within repositories that have both, using each repository’s median for each kind of commit and a Wilcoxon signed-rank test. The three planned tests of the association between AGENTS.md and window commits, hours with commits and test files form one family with Holm correction; the other tests stand alone. We chose AGENTS.md as the predictor because it was the most common agent trace and is written to instruct an agent, so it marks teams that prepared their repository for agent work, and the three outcomes because they describe the amount of work, its spread over the event and one engineering practice. We fixed the planned tests after exploring descriptive numbers and before running them. The plan, the deviations from it and their reasons are released with the data, and analyses added after the plan are labelled *exploratory*.

4 Results

4.1 Population and participation (RQ1)

Most of the gap between the repositories the organisers created and those that received work during the event lies in repositories that the organisers themselves closed before the event or created the evening before it. The organisers created 3,649 repositories, but not all of them were open during the event (Figure 2a). Of these, 446 (12.2%) had been archived before it began, mostly on three days: 126 repositories between 18:30 and 18:32 on 15 September, 56 on 17 September, and 242 between 20:01 and 20:05 on 22 September, with 22 more on other days. Most of them had been created between 15 September and 21 September, and the archiving on 22 September began minutes after the evening batch of 1,040 new repositories was created. None of the repositories archived before the event became active, although, in an exploratory comparison, early work was far more common among them: 157 of the 446 (35.2%) already held team commits from before the event, against 210 of the 3,203 open repositories (6.6%). Of the batch, only 14 repositories became active.

Over all repositories, 29.0% were active, against 33.0% of the 3,203 repositories open during the event and 48.2% of those that were open and not in the batch (Table 3). Within this last group, activity varied only weakly with the creation date (Figure 2b), from 43.2% for repositories created on 22 September to 53.9% for those created between 1 September

Table 3: Main estimates of the study. n is the number of repositories with the property and N the denominator (all repositories, those open and not in the batch, the evening batch, or active repositories; written READMEs; or repositories whose README names the tool), with 95% Wilson intervals.

Quantity	n / N	% (95% CI)
<i>Population and participation</i>		
Active, of all repositories	1,057 / 3,649	29.0 (27.5–30.5)
Active, of repositories open during the event and not in the batch	1,043 / 2,163	48.2 (46.1–50.3)
Active, of the evening batch	14 / 1,040	1.3 (0.8–2.2)
<i>Work over the event window (active repositories)</i>		
Most window commits in the last hour	163 / 1,057	15.4 (13.4–17.7)
Commits in at least four of the five hours	838 / 1,057	79.3 (76.7–81.6)
<i>Traces of coding agents (active repositories)</i>		
Trace specific to Codex	229 / 1,057	21.7 (19.3–24.2)
Codex trace including AGENTS.md	444 / 1,057	42.0 (39.1–45.0)
Claude Code trace	214 / 1,057	20.2 (17.9–22.8)
Codex signature, README names Codex	4 / 120	3.3 (1.3–8.3)
Claude signature, README names Claude Code	30 / 41	73.2 (58.1–84.3)
<i>What teams built (active repositories; READMEs: written READMEs)</i>		
Test files	920 / 1,057	87.0 (84.9–88.9)
LLM SDK or API	772 / 1,057	73.0 (70.3–75.6)
README in Russian	910 / 1,033	88.1 (86.0–89.9)
README in Kazakh	22 / 1,033	2.1 (1.4–3.2)
<i>Credential exposure (active repositories)</i>		
Format-specific strings in history	73 / 1,057	6.9 (5.5–8.6)
OpenAI key strings in history	61 / 1,057	5.8 (4.5–7.3)
OpenAI key strings still present	30 / 1,057	2.8 (2.0–4.0)

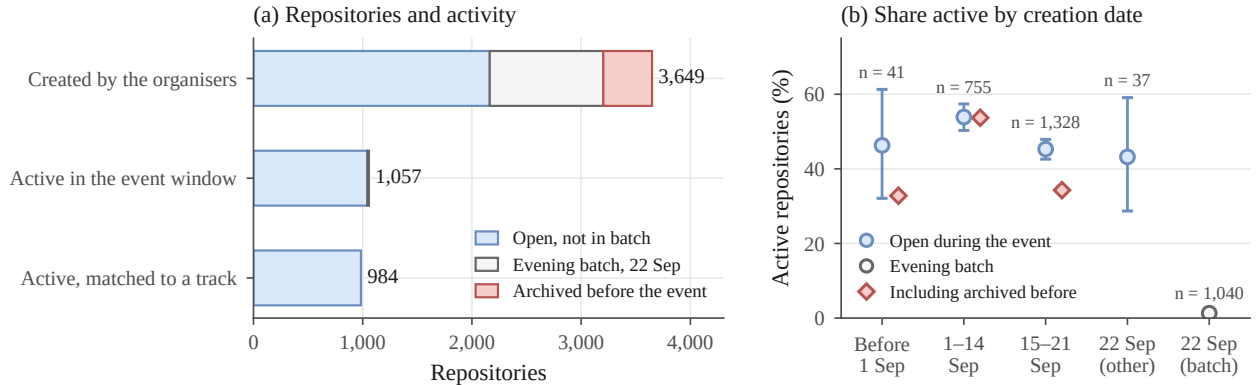


Figure 2: Population. (a) Repositories created by the organisers, split into those open during the event and not in the evening batch, the batch, and those archived before the event; below them, the active repositories (the 14 of the batch form the thin grey segment) and those matched to a track. (b) Share of repositories that became active by creation date, with 95% Wilson intervals, for repositories open during the event (circles; n counts them) and for all repositories including those archived before the event (diamonds, without intervals, shown only where repositories of the group were archived before the event). The two repositories created on the event day are omitted.

and 14 September ($\chi^2(3) = 14.9$, $p = 0.002$, $V = 0.08$). Among all repositories created between 15 September and 21 September the share was lower (34.3%, against 45.3% of the open ones), and the archiving accounts for the difference. Of the 197 repositories with team commits but none in the window, 194 had all of them before the event and 157 were archived before it.

The window commits of the active repositories have at most 2,740 distinct author identities (median 3 per repository), an upper bound on the number of people because one person can commit under several e-mail addresses; press reports give more than 2,500 participants [46]. Of the active repositories, 93.1% could be assigned to a track.

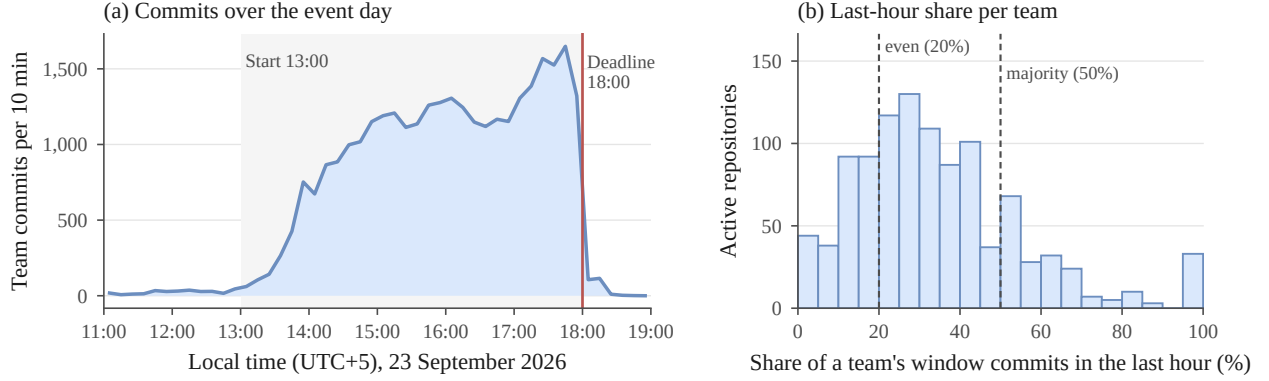


Figure 3: Work over the event day. (a) Team commits per ten minutes, all branches of all repositories; the shaded band is the event window. (b) Distribution over the active repositories of the last-hour share s_i , in bins of five percentage points; the last bin holds the teams that made all their window commits in the last hour. Dashed lines mark an even spread (20%) and a majority (50%).

4.2 Work over the event window (RQ2)

Most teams committed in at least four of the five hours, with a moderate rise towards the deadline rather than a last-minute rush. Active repositories received 30,418 team commits in the window. Activity rose during the first hour and stayed high through the afternoon, and the last hour held 28.8% of all window commits, with the busiest ten minutes at 17:40–17:50 (Figure 3a). The median team made 30.8% of its window commits in the last hour (IQR 19.4–43.8%), above the 20% of an even spread (Figure 3b). A minority of teams (15.4%) made most of their commits in that hour, 33 (3.1%) made all of them in it, while 79.3% committed in at least four of the five hours. Teams made their first window commit a median of 61 minutes after the start (IQR 43–94) and their last a median of 9 minutes before the deadline, and made a median of 19 window commits (IQR 10–35). Some active repositories also had commits from before the event (173) or after the deadline (73).

In an exploratory analysis, k -means with two clusters on the teams’ hourly profiles separated a late group of 323 teams. The silhouette was 0.30, above the mean silhouette of 0.19 (95th percentile 0.20) in 20 simulations without team structure [55], but the two groups differed mainly in their last-hour share, which varies continuously across teams (Figure 3b). We therefore describe the distribution and do not assign teams to types: most worked through the afternoon with a moderate rise towards the deadline, and a minority left most of their commits to the last hour.

4.3 Traces of coding agents (RQ2)

The traces recovered the required agent in a minority of the active repositories, and among teams that named each tool, they recovered another agent, which signs its commits by default, far more completely than the required one. Traces specific to Codex, the required tool, appeared in 229 active repositories (21.7%; Figure 4a). Most were branches whose names start with codex (198 of the 229; 18.7% of active repositories), while authors named “Codex” (3.1%) and commit signatures (1.5%) were rare. An AGENTS.md file appeared in 28.4% of active repositories and was the only Codex-related trace in 215 of them; counting it raises the share with a Codex trace to 42.0%. Next.js had written context files in 89 active repositories (8.4%); had we counted them and the files in vendored folders, as a search by file name would, this share would have been 46.3% and the share with a Claude Code trace 24.9% instead of 20.2%.

AGENTS.md was present in 47.7% of the repositories with a Claude Code trace, against 23.5% of the others. Claude Code, which the rules neither required nor excluded, left a trace in 20.2% of active repositories, mainly through signed commits (16.9%) and CLAUDE.md files (8.5%), and 15.3% had a Claude Code trace but no trace specific to Codex. Other agents appeared in few repositories (Cursor 20, Copilot 5, Gemini 2), and 48.6% of active repositories carried no trace of any agent.

At the level of commits, 11.6% of window commits carried an agent signature or an author named after a tool; 87.6% of these named Claude, and of the 331 that named Codex, 195 did so only through the author name. Pull-request references, a workflow trace, occurred in 35.9% of active repositories.

In an analysis added after the plan, we compared the traces with what teams wrote in their README (Figure 4b). Codex was named in the prose of 120 written READMEs (11.6% of written READMEs), and Claude Code in 41 (4.0%). Among the repositories that named Codex, 32.5% (95% CI 24.8–41.3) carried a trace specific to Codex and 70.0% any Codex trace including AGENTS.md. A signature revealed Codex in 3.3% of them. Among the repositories

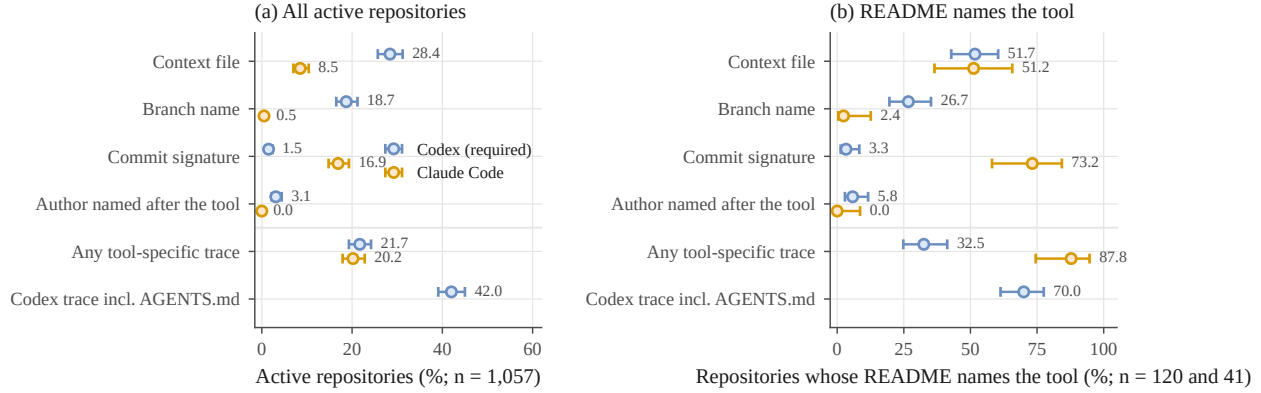


Figure 4: Traces of coding agents. (a) Share of active repositories with each kind of trace, for Codex, which every team had to use, and for Claude Code; for Codex the context file is the shared AGENTS.md, which the tool-specific trace excludes, and the last row adds it; context files that Next.js wrote itself are not counted. That row is not shown for Claude Code, for which it equals the row above. (b) The same traces among the repositories whose README names the tool in its prose. Points are shares with 95% Wilson intervals.

that named Claude Code, 87.8% carried a Claude Code trace and 73.2% a signature. Naming a tool and having any of its traces go together (odds ratio 3.71, 95% CI 2.46–5.59, for Codex including AGENTS.md, and 31.17, 12.54–77.52, for Claude Code, over the active repositories, with those without a written README counted as not naming the tool), so these shares describe detection among teams that name the tool, not the recall of the traces over all users. The broad definition gives the same contrast: a signature revealed Claude in 58.0% of READMEs that contained the word and Codex in 3.0%.

Two further comparisons were exploratory. Agent-signed window commits were not consistently larger than other commits in the same repositories. In the 211 active repositories with both kinds, the per-repository median of signed commits was larger in 58.8% of repositories (Wilcoxon $p = 0.008$), but not among the 155 repositories with at least three signed commits (52.3%, $p = 0.948$) or for Claude-signed commits alone ($p = 0.174$). Pooled over all signed and all other commits of these 211 repositories, the median changes were 129 and 112 lines. Signed commits more often had conventional-commit subjects (48.7% of signed and 43.1% of other non-merge window commits).

In the planned comparison, with the file defined as in Section 3.3 rather than by its name alone as planned, repositories with an AGENTS.md file had more window commits (median 31 vs. 16; $\delta = 0.43$, Holm-adjusted $p < 0.001$), more hours with commits ($\delta = 0.31$, $p < 0.001$) and more often test files (96.0% vs. 83.5%, $V = 0.17$, $p < 0.001$), also after adjusting for the logarithm of one plus the number of window commits (odds ratio 3.30, 95% CI 1.77–6.18). Teams that work through agents may produce both the file and the tests, and teams that prepared more may differ in other ways, so these are associations only.

4.4 What teams built and documented (RQ3)

The typical prototype was a Python or TypeScript application that called an LLM, contained test files and had a README of more than a thousand words, usually in Russian; each of these features held for most active repositories. Python was the primary language of 58.5% of active repositories, followed by TypeScript (21.7%) and JavaScript (12.5%). React (46.5%) was the most common front-end framework and FastAPI (43.2%) the most common back-end framework, with its server Uvicorn in 43.9%; pytest appeared in 39.6%. An LLM SDK or API call appeared in 73.0% of repositories, OpenAI’s in 69.3% and each other provider’s in at most 2.9%. Test files were present in 87.0% of repositories, and, by a stricter measure added after the plan, 85.1% had a test file that defined at least one test; a Dockerfile in 33.3%, a continuous-integration workflow in 14.3% and a deployment configuration in 6.2% (Figure 5). These shares hardly change when repositories with commits before the event or after the deadline are left out (test files 86.6%).

Most written READMEs were in Russian (88.1%), with English second (9.3%) and Kazakh third (2.1%), although Kazakh-specific letters appeared somewhere in 14.2%. Commit subjects, by contrast, were mostly in Latin script (87.5% of non-merge window commits). The median README had 1,645 words (IQR 1,049–2,413) and 16 headings; 97.2% of READMEs gave run or setup instructions, 21.3% included images and 8.1% linked to a deployed application. The primary language (Python, TypeScript, JavaScript or other) differed between tracks (Table 4; $\chi^2(33) = 201.1$, $p < 0.001$, $V = 0.26$, with 8 of 48 cells having expected counts below five), from 30.4% Python in the city-simulator

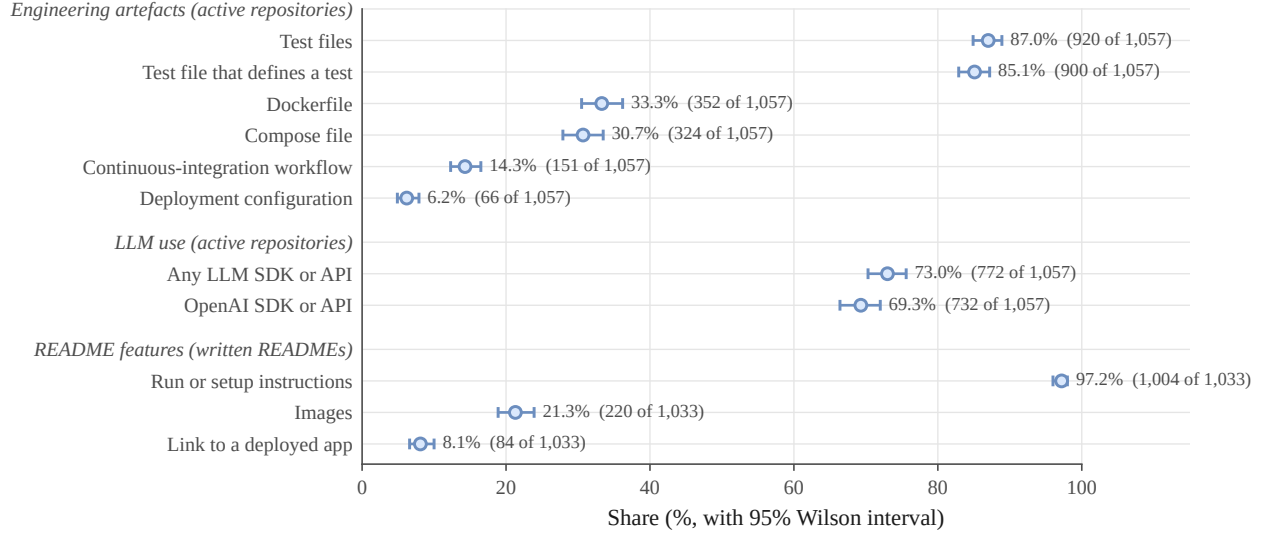


Figure 5: What teams built. Engineering artefacts and LLM use in the 1,057 active repositories, and README features in the 1,033 of them with a written README; shares with 95% Wilson intervals.

track to 95.7% in the telecom track. Window commits per team varied little between tracks ($H(11) = 19.0$, $p = 0.062$, $\epsilon^2 = 0.02$). Kazakh READMEs were rare in every track, including the one whose task was to take minutes from Kazakh, Russian or mixed meeting audio (4.6%).

4.5 How teams solved the same brief (RQ3)

Teams given the same brief chose similar libraries and, in most briefs, one kind of method. In exploratory analyses added after the plan, solutions to the same brief were more alike than solutions to different briefs. The mean Jaccard similarity of the dependency sets was 0.131 for pairs of repositories with the same track and 0.082 for pairs with different tracks (permutation $p = 0.001$, the smallest value that 999 shuffles allow), and most briefs had their own distinctive libraries (Table 5), such as NetworkX for the transaction network, faster-whisper and pyannote.audio for the meeting minutes and scikit-learn for the wind forecast. The excess similarity did not differ clearly between pairs in which both repositories had agent traces (0.053) and pairs in which neither did (0.045; $p = 0.745$). File paths and README headings were also more alike within briefs, though less than dependencies (1.34 and 1.23 times the cross-brief similarity, against 1.60 for dependencies), and for file paths the excess similarity was smaller for pairs with agent traces (0.007 against 0.012; $p = 0.003$), a difference we did not examine further. Of the source files that occurred unchanged in more than one repository, 185 appeared in several tracks and were mostly generated scaffolding of web frameworks, and 28 were confined to one track. The most widely shared of the one-track files were starter code supplied with the finance, telecom and call-routing briefs, and most of the rest belonged to one third-party agent skill found in two repositories of the contractor track; beyond these, identical files gave no sign that teams copied each other’s code.

The coded approaches show the same pattern. The core method differed strongly between briefs ($\chi^2(55) = 2,029.7$, $p < 0.001$, $V = 0.64$; 28 of 72 cells had expected counts below five, 24 of them for the rare pretrained and unclear codes), and in 8 of the 12 briefs one core method was used by at least three quarters of the teams. Most teams used rules or an algorithm for the transaction-network, skills-guide, tariff, replenishment, contractor and budget briefs, a trained model for the wind forecast, and an LLM for briefs built around dialogue or documents, such as task cards, shopping assistance and call routing. Across all briefs, 41.9% of teams let rules or a trained model do the task and used an LLM only to explain or present the result, 35.6% let an LLM produce the main result, 19.6% used no LLM, and 2.9% fitted none of these patterns. Nearly all teams built a web application (94.0%), and 35.1% of READMEs reported a measured result of the solution.

4.6 Credential exposure (RQ4)

Strings in the format of OpenAI API keys reached the public history of 61 active repositories, mostly through environment files committed during the event. Gitleaks’ generic rule produced 4,597 findings in 131 repositories, 93.5% of them in data files (`.json`, `.jsonl`, `.txt` or `.csv`); we treat these as low-precision and do not analyse them further. Format-specific rules produced 127 findings, of which 10 were placeholders. The other 117 lie in 73 active repositories

Table 4: The 12 tracks with the active repositories matched to each, their median number of window commits, and the shares with test files, with Python as primary language and with a Kazakh README (of written READMEs). Partners are named in team READMEs; the briefs are summarised from those READMEs.

	Track	Partner	Brief	Repos	Commits (median)	Tests (%)	Python (%)	Kazakh (%)
T1	Energy	Samruk-Kazyna	Agent that forecasts wind-farm output from weather data	74	25.5	90.5	83.8	2.7
T2	Finance	Freedom	Rebuild an organised group from a transaction network	115	21	87.0	74.8	1.8
T3	Management	Halyk Bank	Guide linking employee skills to growth steps	89	21	93.3	60.7	1.1
T4	Telecom	Beeline	Agent choosing tariff offers for subscribers	46	18	76.1	95.7	0.0
T5	Logistics	ekt.kz	Supplier orders to replenish stock, approved by a manager	80	22.5	91.3	67.5	1.3
T6	Creative industries	Firebird	Choose event contractors and explain the choice	113	16	88.5	63.7	1.8
T7	Education	AI Sana	Turn a business problem into a task for student teams	145	20	86.2	31.0	1.4
T8	Innovation	Samruk-Kazyna	Minutes and action items from Kazakh, Russian or mixed meeting audio	65	17	89.2	87.7	4.6
T9	Communications	Halyk Bank	Voice agent that routes calls in an insurance contact centre	29	31	79.3	72.4	0.0
T10	Trade	ekt.kz	Shopping assistant: products, alternatives, prices, stock	71	19	90.1	49.3	5.6
T11	Special (structure)	Kazakhtelecom	Compare organisation charts before and after a reorganisation	42	20	92.9	61.9	2.4
T12	Special (city)	Astana Innovations	City simulator that splits a budget across districts	115	23	94.8	30.4	0.9

(6.9%), and in 36 repositories (3.4% of the active ones) a string matched by the same rule is still present in the same file in the final version (Figure 6a). Most match the OpenAI API key format: 77 findings in 61 repositories (5.8%), 30 of which still hold such a string. By the author dates that Gitleaks reports, all OpenAI key strings were committed during the event window.

The OpenAI key strings sat mainly in environment files (Figure 6b): 32 findings in `.env.example` and 27 in `.env`. A `.env.example` file is a template meant to be committed, so a real key placed there becomes public even when `.gitignore` excludes `.env`. In the final state, `.gitignore` covered `.env` in 83.6% of active repositories and a `.env.example` file was present in 74.3%. In an exploratory analysis, key strings were not clearly more frequent where the final `.gitignore` covered `.env` (6.1% vs. 4.0%, $V = 0.03$, $p = 0.287$). Coverage went with LLM use (90.9% of repositories with an LLM SDK against 63.9% without), and adjusting for LLM use gave a Mantel–Haenszel odds ratio of 1.03 (95% CI 0.45–2.37, $p = 0.948$). Of the 77 OpenAI findings, 73 could be matched to a commit on a branch, and 5 of these (6.8%) carried an agent signature, against 9.0% of all window commits in the same repositories. Because Codex signatures were rare, this does not show that agents were uninvolved.

Table 5: How teams solved each brief. Main method: the most common core method in the coded approaches and the share of the track’s teams that used it; effective number: the exponential of the Shannon entropy of the method shares; LLM result: teams whose LLM produces the main result; distinctive libraries: the two dependencies most over-represented in the track, among those used by at least a quarter of its teams and at least twice as often as in other tracks, with the share that used them (a dash where none qualifies).

	Track	Main method	Share (%)	Eff. number	LLM result (%)	Distinctive libraries
T1	Energy	trained model	91.9	1.4	2.7	joblib (53%); scikit-learn (74%)
T2	Finance	rules or algorithm	99.1	1.1	0.9	networkx (96%); pyarrow (96%)
T3	Management	rules or algorithm	75.3	2.0	22.5	python-multipart (36%)
T4	Telecom	rules or algorithm	82.6	1.7	15.2	pandas (96%); numpy (91%)
T5	Logistics	rules or algorithm	83.8	1.7	0.0	openpyxl (72%); streamlit (28%)
T6	Creative industries	rules or algorithm	82.3	1.9	15.0	–
T7	Education	LLM prompt	79.3	1.8	94.5	–
T8	Innovation	LLM prompt	47.7	2.8	60.0	pyannote.audio (55%); faster-whisper (77%)
T9	Communications	LLM prompt	65.5	2.1	96.6	pydantic-settings (28%)
T10	Trade	LLM agent	70.4	2.2	88.7	pypdf (28%); python-docx (31%)
T11	Special (structure)	LLM agent	54.8	2.7	78.6	pypdf (38%); python-docx (60%)
T12	Special (city)	rules or algorithm	97.4	1.1	2.6	eslint-config-next (25%); next (31%)

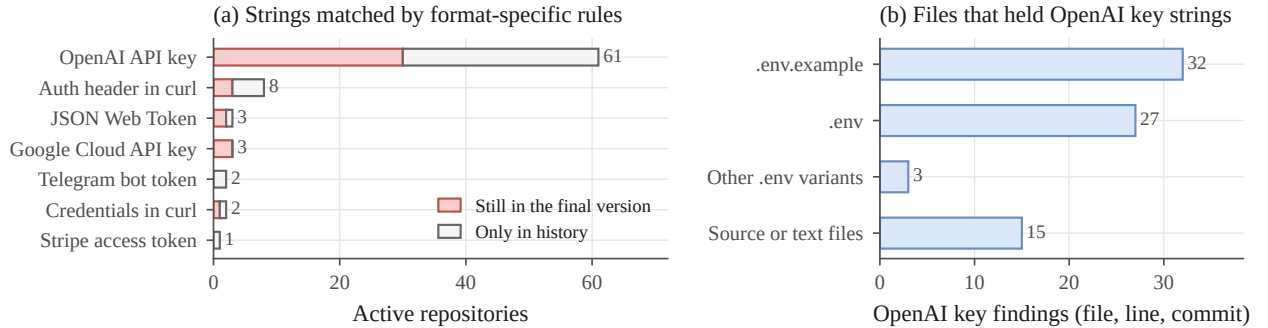


Figure 6: Credential exposure. (a) Active repositories with strings matched by the format-specific rules of Gitleaks, split by whether a string matched by the same rule is still present in the same file in the final version. A repository can appear in more than one row. (b) Files that held the OpenAI key strings; a finding is one file, line and commit, so a key can be counted more than once.

5 Discussion

5.1 Denominators decide rates

The same event yields an activity rate of 29.0%, 33.0% or 48.2%, depending on whether the repositories closed by the organisers and the batch created the evening before are counted. Taken over all repositories, the data suggest that the date of registration mattered a great deal ($\chi^2(3) = 608.2$, $p < 0.001$, $V = 0.41$, with the batch counted with 22 September), whereas among open repositories outside the batch the effect is small ($V = 0.08$). We do not know why the organisers archived repositories in bulk or created the batch; the allocation of limited seats, which the website gave first come, first served [14] and news reports describe [48, 49], and withdrawn or duplicate registrations are possible reasons. Archived repositories also held work from before the event about five times as often as open ones, which fits both teams that tried their repository early and then withdrew and the removal of repositories in which work had started early. Organiser-created repositories record the organisers’ actions as well as the teams’, so archive times, creation bursts and template commits have to be reconstructed before any rate is read as participation. Datasets built from project listings face the opposite problem, since they never contain the empty repositories [10, 18].

5.2 What traces reveal when the tool is known

Every team was told to use Codex, yet traces specific to Codex appeared in about a fifth of the active repositories, and Codex-related traces of any kind in fewer than half, even counting `AGENTS.md`. These data cannot separate compliance from detection, because the observed share is the product of the two; even if every team used Codex, the traces recovered that use in at most 42.0% of the active repositories. Because Codex was required and reads `AGENTS.md`, we take this share as the closer estimate of Codex-related traces, and the 21.7% with traces specific to Codex as a lower bound that does not depend on who wrote the file. Detection also differed sharply between tools. Claude Code adds a co-author trailer to its commits by default [15]; we found no documented default for Codex, whose signatures were rare here even among teams that named it (Table 3). Adoption estimates that rest mainly on commit signatures may therefore count tools that sign by default more completely than tools that do not; running each tool with its default settings in a test repository would check this mechanism directly. The defaults of other software matter as well: Next.js writes the context files of two agents at once, whether or not either agent is used, and it had done so in 8.4% of the active repositories, so studies that count `AGENTS.md` or `CLAUDE.md` files have to separate such files from those that people or agents wrote. The comparison among teams that name each tool also sets two unlike groups side by side. Codex was required, so naming it may state compliance, whereas naming Claude Code was voluntary and may come from heavy users, whose README the agent may have written. The gap in any tool-specific trace (87.8% against 32.5%) is smaller than the gap in signatures, but it mixes the same two effects. Estimates that combine several traces [2] depend less on one tool’s defaults, but they too cannot see use that leaves no trace. Robbes et al. discuss the pitfalls of detecting agents from traces [8]; a known required tool shows how large the gap can be in one setting.

We could not confirm that agent-signed commits are larger than other commits, as Robbes et al. found at scale [2]: the difference depended on which repositories were included. The Claude Code traces also give a lower bound on the use of a tool that the rules did not require. The rules required Codex but did not exclude other tools, so these traces are not breaches; they show that teams also worked with an agent that the rules did not name.

5.3 Work, artefacts and language

Deadline pressure was visible but moderate. The median team made 30.8% of its window commits in the last hour, more than an even spread would give, and most teams committed in at least four of the five hours. Studies of time pressure mostly report higher productivity and lower quality [35], and in programming courses students who start late tend to score lower [34]. Whether the teams that left most of their work to the last hour built weaker prototypes is one of the questions of the pre-registered analysis of the jury’s decisions.

Test files, long READMEs with run instructions and, in a third of repositories, Dockerfiles were produced within five hours. One explanation, which our data cannot test, is that agents make such artefacts cheap to produce, since in many real agent sessions the agent writes nearly all of the committed code [1]. The quality risks of agent-written code are documented elsewhere [4, 5, 41]. Teams given the same brief also converged on similar solutions, and their library overlap was not clearly greater among teams that left agent traces. Early convergence of ideas has also been observed among novices who built software mainly by prompting [13]. A common design let rules or a trained model do the task and an LLM explain the result, which fits briefs that ask for decisions that can be checked. Whether this convergence came from the briefs themselves or from similar tools answering the same brief, we cannot say. The documentation was written mostly in Russian, and Kazakh was rare even in the track built around Kazakh audio. Language models still score lower on Kazakh-language benchmarks than on benchmarks in high-resource languages [56], which may make teams less inclined to work with agents in Kazakh, but the audience that teams wrote for may matter as much, and our data cannot separate the two.

5.4 Implications

For organisers, creating repositories after check-in would turn the GitHub organisation into a record of participants rather than of registrations. Because many of the exposed keys sat in `.env.example`, a file meant to be committed, simple measures could have closed this route. GitHub blocks pushes of OpenAI API keys to public repositories by default, but the block can be bypassed [44, 45], and we cannot tell whether the strings found here were pushed past it, pushed while it did not apply, or were not live keys. Repository templates that ship a `.gitignore` and a placeholder-only `.env.example`, push protection at the level of the organisation, which is off by default and turns bypasses into alerts [45], and short-lived keys issued for the event would all help, in line with earlier recommendations [36, 39]. For researchers who measure agent use from repositories, our results suggest reporting how the population was formed, separating traces that are specific to one tool from files that several tools may read or that frameworks generate, reporting each trace for each tool, and checking trace-based estimates, where possible, against a reference such as a known required tool or what developers state.

6 Threats to validity and limitations

We organise the threats following the empirical standards for repository mining [57].

Construct validity. Author identities overstate the number of people. Commit counts measure activity, not effort or quality, and test files show an intention to test, not working tests. Agent traces are partial by design. `AGENTS.md` is read by several agents, and our rule for the files that Next.js writes recognises only its current managed block, so files from other generators may still be counted. An author named “Codex” is a trace whose origin we could not verify. Naming a tool in a README does not show how much the tool was used, and it means different things for a required and a voluntary tool. We therefore report what each kind of trace shows and treat none of them as a measure of use. The README language rule is a letter heuristic that misses Kazakh typed with Russian or Latin letters, and its errors lay mainly in READMEs it called Kazakh or mixed (Section 3.5), so the Kazakh share may be slightly overstated. The coded approaches rest on what the README states and on one hosted model at one date, and they were not checked against hand labels; the dependency sets rest on declared manifests. Credential findings match key formats, were not tested, and depend on a placeholder filter that was not validated against hand labels.

Internal validity and reliability. The event window is inferred rather than taken from a schedule, but it is supported by the archive times, and shifting it changes the number of active repositories little (Table 2). Features are measured at the final state, which in some repositories includes commits from before the event or after the deadline; leaving those repositories out barely changes the shares. The `.gitignore` rules are also measured at the final state, not at the time a key was committed. Reading bulk archiving from last-update times is an inference, consistent with the absence of any commit in those repositories during the event. The associations with `AGENTS.md` and `.gitignore` are observational, and the README-naming and commit-size analyses were added after the analysis plan. Track labels come from keyword rules on README text, so teams that did not describe their task stay unassigned, and the labels and the README language sample were checked by one person. The default-branch commit at which each clone ended is released, so that the data can be collected again.

External validity. We study one in-person event in one country, with one required agent and a five-hour window. Other events differ in duration, rules, tools and participants, the default behaviour of agents changes between versions, and the share of empty repositories depends on how an organiser provisions them. The participants were adults working or studying in IT or with projects of their own, so the results describe technically literate teams, not developers in general.

What the study cannot show. The data record traces, not use. We cannot say how many teams used Codex, how much of the code any agent wrote, or whether agents improved or harmed the prototypes. A repository stands for a registration, not a person, so we make no claims about individuals, their skills or what they learned. Nearly all comparisons are observational, several were added after the analysis plan, and they describe one event; they can guide controlled studies but cannot replace them.

7 Conclusions

A hackathon at which the organisers created the repositories and named the required tool allows repository measures to be checked against what was known in advance. At HackAlem AI, the choice of denominator moved the activity rate by 19.2 percentage points, because the organisers archived repositories and created a batch before the event. Traces specific to the required agent appeared in about a fifth of the active repositories, and Codex-related traces of any kind in fewer than half, even counting `AGENTS.md`, which several agents read; a web framework had written such files itself in 8.4% of the active repositories. In an exploratory comparison among teams that named each tool, signatures revealed Claude Code, which signs its commits by default, far more often than Codex. Teams given the same brief used more similar libraries and, by the coded approaches, mostly one kind of method; their library overlap was not clearly greater among teams that left agent traces. OpenAI key strings became public mainly through committed `.env.example` and `.env` files. Studies of AI coding agents that start from repositories should therefore report how their population was formed and what each trace can recover. Several questions remain open. After the awards, we will test the pre-registered hypotheses about the jury’s decisions and compare these repositories with hackathon repositories from before coding agents [11]. Running each agent with its default settings would show which traces it leaves; running the teams’ tests and hand-labelling a sample of approaches would check the test files and the coded approaches.

Author contributions

Talgar Bayan conceived and designed the study, collected and analysed the data, and wrote and revised the paper.

Funding

This research received no external funding.

Institutional review board statement

Not applicable. The study used only public repositories and public web pages, involved no interaction with human participants, and accessed no private system. Personal data and credentials were handled as described in Section 3.4, and the organisers were informed of the exposed credentials before publication.

Informed consent statement

Not applicable.

Data availability statement

The pseudonymised dataset, the pipeline, the analysis plan and its deviations are available at <https://github.com/talgarbayan/HackAlemAI-Repo-Data-Mining> and archived on Zenodo as release v1.0.4 (doi:10.5281/zenodo.23091476). The released data use random identifiers; because the source repositories are public, this is pseudonymisation and not anonymisation. Credential findings are released only as aggregates. The analysis of the jury’s decisions was pre-registered on the Open Science Framework before any finalist or winner was announced (<https://osf.io/bm3je>), following a template for secondary data analysis [58].

Acknowledgments

The author thanks Nazarbayev University for the computer, workspace and research infrastructure used in this study, and SPG for its support. The author found the organiser-created repositories, had the idea of mining and analysing them, and designed the study, its research questions and its methods, including how the data were collected and how each analysis script should work; the author also wrote the prompts and guided every step. Claude Code (Anthropic; Claude Opus 5.5) was used to write and run the scripts, which sped up data collection and analysis, and to draft and edit the text. The author checked all results, numbers and references and takes full responsibility for the content.

Conflicts of interest

The author declares no conflicts of interest. The author is not affiliated with the organisers of HackAlem AI, and this paper reports no official results of the event.

Abbreviations

API, application programming interface; CI, confidence interval; IQR, interquartile range; LLM, large language model; SDK, software development kit.

References

- [1] Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. SWE-chat: Coding Agent Interactions From Real Users in the Wild. arXiv preprint arXiv:2604.20779, doi:10.48550/arXiv.2604.20779, 2026.
- [2] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. Agentic Much? Adoption of Coding Agents on GitHub. *ACM Transactions on Software Engineering and Methodology*, June 2026. doi:10.1145/3822180.

- [3] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. *ACM Transactions on Software Engineering and Methodology*, September 2026. doi:[10.1145/3840295](https://doi.org/10.1145/3840295).
- [4] Shyam Agarwal, Hao He, and Bogdan Vasilescu. AI IDEs or Autonomous Agents? Measuring the Impact of Coding Agents on Software Development. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR '26, pages 857–862. ACM, April 2026. doi:[10.1145/3793302.3793589](https://doi.org/10.1145/3793302.3793589).
- [5] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR '26, pages 181–193. ACM, April 2026. doi:[10.1145/3793302.3793349](https://doi.org/10.1145/3793302.3793349).
- [6] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, MSR 2014, pages 92–101. ACM, May 2014. doi:[10.1145/2597073.2597074](https://doi.org/10.1145/2597073.2597074).
- [7] Sebastian Baltes and Paul Ralph. Sampling in software engineering research: a critical review and guidelines. *Empirical Software Engineering*, 27(4):94, April 2022. doi:[10.1007/s10664-021-10072-8](https://doi.org/10.1007/s10664-021-10072-8).
- [8] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. Promises, Perils, and (Timely) Heuristics for Mining Coding Agent Activity. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR '26, pages 496–507. ACM, April 2026. doi:[10.1145/3793302.3793375](https://doi.org/10.1145/3793302.3793375).
- [9] Connie W. Chau and Elizabeth M. Gerber. On Hackathons: A Multidisciplinary Literature Review. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, pages 1–21. ACM, April 2023. doi:[10.1145/3544548.3581234](https://doi.org/10.1145/3544548.3581234).
- [10] Ahmed Imam, Tapajit Dey, Alexander Nolte, Audris Mockus, and James D. Herbsleb. The Secret Life of Hackathon Code: Where does it come from and where does it go? In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pages 68–79. IEEE, May 2021. doi:[10.1109/msr52588.2021.00020](https://doi.org/10.1109/msr52588.2021.00020).
- [11] Sjoerd Halmans, Lavinia Paganini, Alexander Serebrenik, and Alexander Nolte. HackRep: A Large-Scale Dataset of GitHub Hackathon Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR '26, pages 579–583. ACM, April 2026. doi:[10.1145/3793302.3793305](https://doi.org/10.1145/3793302.3793305).
- [12] Wangyiyao Zhou, Alexander Serebrenik, and Alexander Nolte. Quick Build, Careful Check? Generative AI Use in Hackathons. In *Proceedings of the 2026 International Conference on Game Jams, Hackathons and Game Creation Events*, ICGJ 2026, pages 23–27. ACM, August 2026. doi:[10.1145/3833089.3833091](https://doi.org/10.1145/3833089.3833091).
- [13] Kiev Gama, Filipe Calegario, Victoria Jackson, Alexander Nolte, Luiz Morais, and Vinicius Cardoso Garcia. “Can you feel the vibes?”: An exploration of novice programmer engagement with vibe coding. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, ICSE-SEET '26, pages 485–496. ACM, April 2026. doi:[10.1145/3786580.3786992](https://doi.org/10.1145/3786580.3786992).
- [14] HackAlem AI. HackAlem AI — Astana, 23.09.2026, 2026. URL <https://hackalem.ai/>. Official event website. Accessed 24 and 29 September 2026.
- [15] Anthropic. Claude Code settings reference: git and attribution, 2026. URL <https://code.claude.com/docs/en/settings-reference>. Accessed 27 September 2026.
- [16] Alexander Nolte, Irene-Angelica Chounta, and James D. Herbsleb. What Happens to All These Hackathon Projects? Identifying Factors to Promote Hackathon Project Continuation. *Proceedings of the ACM on Human-Computer Interaction*, 4(CSCW2):1–26, October 2020. doi:[10.1145/3415216](https://doi.org/10.1145/3415216).
- [17] Lukas McIntosh and Caroline D. Hardin. Do Hackathon Projects Change the World? An Empirical Analysis of GitHub Repositories. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*, SIGCSE '21, pages 879–885. ACM, March 2021. doi:[10.1145/3408877.3432435](https://doi.org/10.1145/3408877.3432435).
- [18] Ahmed Samir Imam Mahmoud, Tapajit Dey, Alexander Nolte, Audris Mockus, and James D. Herbsleb. One-off events? An empirical study of hackathon code creation and reuse. *Empirical Software Engineering*, 27(7):167, September 2022. doi:[10.1007/s10664-022-10201-x](https://doi.org/10.1007/s10664-022-10201-x).
- [19] Christian Bird, Peter C. Rigby, Earl T. Barr, David J. Hamilton, Daniel M. German, and Prem Devanbu. The promises and perils of mining git. In *2009 6th IEEE International Working Conference on Mining Software Repositories*, pages 1–10. IEEE, May 2009. doi:[10.1109/msr.2009.5069475](https://doi.org/10.1109/msr.2009.5069475).

- [20] Nicole Hoess, Carlos Paradis, Rick Kazman, and Wolfgang Maurer. Does the Tool Matter? Exploring Some Causes of Threats to Validity in Mining Software Repositories. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 645–656. IEEE, March 2025. doi:[10.1109/saner64311.2025.00067](https://doi.org/10.1109/saner64311.2025.00067).
- [21] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabic, Massimiliano Di Penta, and Gabriele Bavota. Unveiling ChatGPT’s Usage in Open Source Projects: A Mining-based Study. In *Proceedings of the 21st International Conference on Mining Software Repositories, MSR ’24*, pages 571–583. ACM, April 2024. doi:[10.1145/3643991.3644918](https://doi.org/10.1145/3643991.3644918).
- [22] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv preprint [arXiv:2507.15003](https://arxiv.org/abs/2507.15003), 2025. doi:[10.48550/arXiv.2507.15003](https://doi.org/10.48550/arXiv.2507.15003).
- [23] OpenAI. Custom instructions with AGENTS.md (Codex documentation), 2026. URL <https://learn.chatgpt.com/docs/agent-configuration/agents-md>. Accessed 27 September 2026.
- [24] AGENTS.md. AGENTS.md: a simple, open format for guiding coding agents, 2026. URL <https://agents.md/>. Accessed 28 September 2026.
- [25] Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv preprint [arXiv:2602.11988](https://arxiv.org/abs/2602.11988), doi:[10.48550/arXiv.2602.11988](https://doi.org/10.48550/arXiv.2602.11988), 2026.
- [26] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-world GitHub Issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html. Preprint: doi:[10.48550/arXiv.2310.06770](https://doi.org/10.48550/arXiv.2310.06770).
- [27] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37, NeurIPS 2024*, pages 50528–50652. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi:[10.52202/079017-1601](https://doi.org/10.52202/079017-1601).
- [28] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv preprint [arXiv:2302.06590](https://arxiv.org/abs/2302.06590), doi:[10.48550/arXiv.2302.06590](https://doi.org/10.48550/arXiv.2302.06590), 2023.
- [29] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Measuring GitHub Copilot’s Impact on Productivity. *Communications of the ACM*, 67(3):54–63, February 2024. doi:[10.1145/3633453](https://doi.org/10.1145/3633453).
- [30] Ashley J. Chen, Yijia Cao, Minghao Shao, Ramesh Karri, and Muhammad Shafique. Code for All: Educational Applications of the “Vibe Coding” Hackathon in Programming Education across All Skill Levels. arXiv preprint [arXiv:2604.22747](https://arxiv.org/abs/2604.22747), doi:[10.48550/arXiv.2604.22747](https://doi.org/10.48550/arXiv.2604.22747), 2026.
- [31] Ramteja Sajja, Carlos Erazo Ramirez, Zhouyuan Li, Bekir Z. Demiray, Yusuf Sermet, and Ibrahim Demir. Integrating Generative AI in Hackathons: Opportunities, Challenges, and Educational Implications. *Big Data and Cognitive Computing*, 8(12):188, December 2024. doi:[10.3390/bdcc8120188](https://doi.org/10.3390/bdcc8120188).
- [32] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. Computing Education in the Era of Generative AI. *Communications of the ACM*, 67(2):56–67, January 2024. doi:[10.1145/3624720](https://doi.org/10.1145/3624720).
- [33] Maëlick Claes, Mika V. Mäntylä, Miikka Kuutila, and Bram Adams. Do programmers work at night or during the weekend? In *Proceedings of the 40th International Conference on Software Engineering, ICSE ’18*, pages 705–715. ACM, May 2018. doi:[10.1145/3180155.3180193](https://doi.org/10.1145/3180155.3180193).
- [34] Ayaan M. Kazerouni, Stephen H. Edwards, and Clifford A. Shaffer. Quantifying Incremental Development Practices and Their Relationship to Procrastination. In *Proceedings of the 2017 ACM Conference on International Computing Education Research, ICER ’17*, pages 191–199. ACM, August 2017. doi:[10.1145/3105726.3106180](https://doi.org/10.1145/3105726.3106180).
- [35] Miikka Kuutila, Mika Mäntylä, Umar Farooq, and Maëlick Claes. Time pressure in software engineering: A systematic review. *Information and Software Technology*, 121:106257, May 2020. doi:[10.1016/j.infsof.2020.106257](https://doi.org/10.1016/j.infsof.2020.106257).
- [36] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories. In *Proceedings 2019 Network and Distributed System Security Symposium, NDSS 2019*. Internet Society, 2019. doi:[10.14722/ndss.2019.23418](https://doi.org/10.14722/ndss.2019.23418).

- [37] Setu Kumar Basak, Lorenzo Neil, Bradley Reaves, and Laurie Williams. SecretBench: A Dataset of Software Secrets. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 347–351. IEEE, May 2023. doi:[10.1109/msr59073.2023.00053](https://doi.org/10.1109/msr59073.2023.00053).
- [38] Setu Kumar Basak, Jamison Cox, Bradley Reaves, and Laurie Williams. A Comparative Study of Software Secrets Reporting by Secret Detection Tools. In *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–12. IEEE, October 2023. doi:[10.1109/esem56168.2023.10304853](https://doi.org/10.1109/esem56168.2023.10304853).
- [39] Alexander Krause, Jan H. Klemmer, Nicolas Huaman, Dominik Wermke, Yasemin Acar, and Sascha Fahl. Pushed by Accident: A Mixed-Methods Study on Strategies of Handling Secret Information in Source Code Repositories. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2527–2544, Anaheim, CA, August 2023. USENIX Association. ISBN 978-1-939133-37-3. URL <https://www.usenix.org/conference/usenixsecurity23/presentation/krause>. Preprint: doi:[10.48550/arXiv.2211.06213](https://doi.org/10.48550/arXiv.2211.06213).
- [40] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. Do Users Write More Insecure Code with AI Assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS ’23*, pages 2785–2799. ACM, November 2023. doi:[10.1145/3576915.3623157](https://doi.org/10.1145/3576915.3623157).
- [41] Songwen Zhao, Danqing Wang, Kexun Zhang, Jiaxuan Luo, Zhuo Li, and Lei Li. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks. arXiv preprint arXiv:2512.03262, doi:[10.48550/arXiv.2512.03262](https://doi.org/10.48550/arXiv.2512.03262), 2025.
- [42] Pinran Gao, Lingxiang Wang, Yi Liu, Kunpeng Liu, Fan Yang, and Ying Zhang. Mind your key: An Empirical Study of LLM API Credential Leakage in iOS Apps. arXiv preprint arXiv:2606.12212, doi:[10.48550/arXiv.2606.12212](https://doi.org/10.48550/arXiv.2606.12212), 2026.
- [43] GitHub. About secret scanning for partners (GitHub docs), 2026. URL <https://docs.github.com/en/code-security/secret-scanning/introduction/about-secret-scanning-for-partners>. Accessed 28 September 2026.
- [44] GitHub. Supported secret scanning patterns (GitHub docs), 2026. URL <https://docs.github.com/en/code-security/secret-scanning/introduction/supported-secret-scanning-patterns>. Accessed 28 September 2026.
- [45] GitHub. Push protection (GitHub docs), 2026. URL <https://docs.github.com/en/code-security/concepts/secret-security/push-protection>. Accessed 28 September 2026.
- [46] The Astana Times. Astana stages massive AI hackathon in Guinness World Records bid, September 2026. URL <https://astanatimes.com/2026/09/astana-stages-massive-ai-hackathon-in-guinness-world-records-bid/>. Accessed 24 September 2026.
- [47] Kazinform. What is built in five hours: how the largest AI hackathon in Astana is run, September 2026. URL <https://www.inform.kz/ru/chto-sozdayut-za-pyat-chasov-kak-prohodit-krupneyshiy-ai-hakaton-v-astane-72e380f9>. In Russian. Accessed 24 September 2026.
- [48] Ulys Media. “Survival simulator”: participants criticised HackAlem AI in Astana, September 2026. URL <https://ulysmedia.kz/news/81431-simuliator-vyzhivaniia-uchastniki-raskritikovali-hackalem-ai-v-astane/>. In Russian. Accessed 24 September 2026.
- [49] Qumash.kz. Five hours in the queue: HackAlem AI participants in Astana complained about the organisation, September 2026. URL <https://qumash.kz/news/pyat-chasov-v-ocheredi-uchastniki-hackalem-ai-v-astane-pozhalovalis-na-organizatsiyu/>. In Russian. Accessed 24 September 2026.
- [50] Vercel. How to set up your Next.js project for AI coding agents (Next.js documentation, version 16.3.6), 2026. URL <https://nextjs.org/docs/app/guides/ai-agents>. Last updated 7 September 2026. Accessed 29 September 2026.
- [51] Anthropic. How Claude remembers your project (Claude Code documentation), 2026. URL <https://code.claude.com/docs/en/memory>. Accessed 28 September 2026.
- [52] Gede Artha Azriadi Prana, Christoph Treude, Ferdian Thung, Thushari Atapattu, and David Lo. Categorizing the Content of GitHub README Files. *Empirical Software Engineering*, 24(3):1296–1327, October 2019. doi:[10.1007/s10664-018-9660-3](https://doi.org/10.1007/s10664-018-9660-3).
- [53] Gitleaks contributors. Gitleaks: a tool for detecting secrets in git repositories, version 8.30.1, 2026. URL <https://github.com/gitleaks/gitleaks>. Release v8.30.1, 21 March 2026. Accessed 24 September 2026.
- [54] Maciej Tomczak and Ewa Tomczak. The need to report effect size estimates revisited. An overview of some recommended measures of effect size. *Trends in Sport Sciences*, 21(1):19–25, 2014. URL <https://tss.awf.poznan.pl/The-need-to-report-effect-size-estimates-revisited-An-overview-of-some-recommended,188960,0,2.html>.

- [55] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, November 1987. doi:[10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7).
- [56] Mukhammed Togmanov, Nurdaulet Mukhituly, Diana Turmakhan, Jonibek Mansurov, Maiya Goloburda, Akhmed Sakip, Zhuohan Xie, Yuxia Wang, Bekassyl Syzdykov, Nurkhan Laiyk, Alham Fikri Aji, Ekaterina Kochmar, Preslav Nakov, and Fajri Koto. KazMMLU: Evaluating Language Models on Kazakh, Russian, and Regional Knowledge of Kazakhstan. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14403–14416. Association for Computational Linguistics, 2025. doi:[10.18653/v1/2025.acl-long.701](https://doi.org/10.18653/v1/2025.acl-long.701).
- [57] Paul Ralph, Nauman bin Ali, Sebastian Baltes, Domenico Bianculli, Jessica Diaz, Yvonne Dittrich, Neil Ernst, Michael Felderer, Robert Feldt, Antonio Filieri, Breno Bernard Nicolau de França, Carlo Alberto Furia, Greg Gay, Nicolas Gold, Daniel Graziotin, Pinjia He, Rashina Hoda, Natalia Juristo, Barbara Kitchenham, Valentina Lenarduzzi, Jorge Martínez, Jorge Melegati, Daniel Mendez, Tim Menzies, Jefferson Moller, Dietmar Pfahl, Romain Robbes, Daniel Russo, Nyti Saarimäki, Federica Sarro, Davide Taibi, Janet Siegmund, Diomidis Spinellis, Mirosław Staron, Klaas Stol, Margaret-Anne Storey, Damian Tamburri, Marco Torchiano, Christoph Treude, Burak Turhan, Xiaofeng Wang, and Sira Vegas. Empirical Standards for Software Engineering Research. arXiv preprint arXiv:2010.03525, doi:[10.48550/arXiv.2010.03525](https://doi.org/10.48550/arXiv.2010.03525), 2020.
- [58] Olmo R. Van den Akker, Sara Weston, Lorne Campbell, Bill Chopik, Rodica Damian, Pamela Davis-Kean, Andrew Hall, Jessica Kosie, Elliott Kruse, Jerome Olsen, Stuart Ritchie, KD Valentine, Anna Van ’t Veer, and Marjan Bakker. Preregistration of secondary data analysis: A template and tutorial. *Meta-Psychology*, 5, November 2021. doi:[10.15626/mp.2020.2625](https://doi.org/10.15626/mp.2020.2625).